# CodeLab

Online Compiler (OneCompiler Embed)

Software Documentation Suite

| | |
|---|---|
| Version: | 1.1 |
| Date: | 2026-10-11 |
| Author: | Generated from Repository Analysis |
| Stack: | Next.js 16.4.0, React 19.3.0, TypeScript, Tailwind CSS 4, lucide-react, OneCompiler Embed (iframe) |

**Confidentiality:** This document is generated strictly from the repository listed below. No customers, metrics, APIs, or features outside the repository are claimed. Repository: C:/Dev/codelab (Next.js App Router application at repository root)

# Contents

**1 Executive Summary** **1**
1.1 Problem and solution . . . 1
1.2 Technology and architecture . . . 1
1.3 Maturity and limitations . . . 1

**2 Project Overview** **1**
2.1 Identity . . . 2
2.2 Repository inventory . . . 2
2.3 Explicit non-goals (not in repository) . . . 2

**3 Product Description** **2**
3.1 Product identity . . . 3
3.2 Problem and solution . . . 3
3.3 Target audience and personas . . . 3
3.4 Features (implemented) . . . 3
3.5 Non-features . . . 3
3.6 Visual identity and SEO keywords . . . 4

**4 Business Requirements Document (BRD)** **4**
4.1 Business problem . . . 4
4.2 Business objectives (as evidenced) . . . 4
4.3 Stakeholders and user groups . . . 4
4.4 Success metrics, constraints, risks . . . 4

**5 Product Requirements Document (PRD)** **5**
5.1 Vision and goals . . . 5
5.2 Personas and user stories . . . 5
5.3 Scope . . . 5
5.4 Acceptance criteria (verifiable) . . . 5
5.5 Constraints, dependencies, risks . . . 5

**6 Functional Requirements** **6**

**7 Non-Functional Requirements** **7**

**8 Technical Requirements** **7**
8.1 Explicitly absent . . . 8

**9 Software and System Architecture** **8**
9.1 System context . . . 8
9.2 High-level design . . . 9
9.3 Low-level design . . . 9
9.4 Backend, database, and API . . . 9

**10 Frontend Architecture** **9**
10.1 Route tree . . . 9
10.2 Component catalogue . . . 10
10.3 Props contract . . . 10
10.4 Styling approach . . . 10
10.5 State and data fetching . . . 10

**11 Integrations** **10**
11.1 OneCompiler embed . . . 11
11.2 Resilience notes . . . 11

**12 User Flows** **11**
12.1 Flow 1 — run code immediately . . . 11
12.2 Flow 2 — learn a language via a guide . . . 11
12.3 Flow 3 — teacher assigns, student submits by link . . . 11
12.4 Flow 4 — answer-seeking and support . . . 12

**13 Data Flows** **12**
13.1 Build-time data flow . . . 12
13.2 Runtime data flow . . . 12
13.3 Third-party data exposure . . . 12

**14 Configuration** **12**
14.1 Environment variables . . . 13

**15 UI/UX** **13**
15.1 Screens . . . 13
15.2 Layout and responsive behavior . . . 13
15.3 States and validation . . . 13
15.4 Accessibility . . . 14

**16 Design System** **14**

**17 Security** **14**

**18 Performance and Scalability** **15**
18.1 Performance . . . 15
18.2 Scalability . . . 16

**19 Testing** **16**
19.1 Current state . . . 16
19.2 Recommended strategy (RECOMMENDED, not implemented) . . . 16

**20 Deployment** **16**
20.1 Prerequisites . . . 16
20.2 Build and hosting . . . 17

**21 CI/CD** **17**
21.1 Continuous integration . . . 17
21.2 Supporting automation . . . 17

**22 SEO** **17**
22.1 Current implementation . . . 17
22.2 Known limitations . . . 18

**23 AEO (Answer Engine Optimization)** **18**
23.1 Answer-first content . . . 18
23.2 Question coverage . . . 18

**24 GEO (Generative Engine Optimization) 18**
24.1 Entity graph . . . 18
24.2 Citation-friendly properties . . . 18

**25 Known Issues 18**

**26 Technical Debt 19**

**27 Roadmap 20**
27.1 Existing (EXISTING) . . . 20
27.2 Recommended next (RECOMMENDED) . . . 20
27.3 Potential future work (POTENTIAL) . . . 20

**28 Development Guidelines 21**
28.1 Setup and commands . . . 21
28.2 Conventions . . . 21
28.3 Definition of done . . . 21

**29 Contribution Guide 21**
29.1 Workflow . . . 21
29.2 Conventions for contributors . . . 22

**30 Glossary 22**

# List of Tables

1 Project identity (evidence-based) . . . 2
2 Implemented features with evidence . . . 3
3 Stakeholders (evidence-based) . . . 4
4 Functional requirements traceable to code . . . 6
5 Non-functional requirements and their implementation state . . . 7
6 Technology stack (actual versions) . . . 8
7 Technologies not present (verified by file search) . . . 8
8 Components (exhaustive) . . . 10
9 OneCompiler integration fact sheet . . . 11
10 Configuration files and their role . . . 13
11 Environment variables . . . 13
12 Actual design values in use . . . 14
13 Security review (evidence-based) . . . 15
14 SEO signals present in the repo . . . 17
15 Verified issues . . . 19
16 Technical debt register . . . 20
17 Glossary (repository-scoped) . . . 22

# 1 Executive Summary

**Status:** IMPLEMENTED (what exists) + INFERRED (interpretation, marked inline).

CodeLab is a multi-route marketing and documentation website wrapped around a live online compiler. **Status:** IMPLEMENTED. Evidence: 17 static routes plus 12 programmatic language guides and 4 blog posts under `src/app/`; the `/compiler` route embeds the OneCompiler editor via `CodeEditor` (`src/components/CodeEditor.tsx`, `iframe` pointed at `https://onecompiler.com/embed`).

## 1.1 Problem and solution

Users who want to try, learn, or share code without installing toolchains need an instantly available editor plus trustworthy guidance. **Status:** INFERRED. The solution (IMPLEMENTED) pairs a zero-install playground with server-rendered guides (per-language pages, stdin tutorials, use-case workflows), so both humans and crawlers get real content while execution stays delegated to the OneCompiler embed.

## 1.2 Technology and architecture

Next.js 16.4.0 (App Router), React 19.3.0, TypeScript (strict), Tailwind CSS 4, `lucide-react` icons. **Status:** IMPLEMENTED. Evidence: `package.json`, `tsconfig.json`, `src/app/globals.css`. There is no backend, database, API route, authentication, test suite, Docker image, or environment configuration in the repository. A lint+build CI workflow exists under `.github/workflows/ci.yml`. **Status:** IMPLEMENTED (verified by file enumeration on 2026-10-11).

## 1.3 Maturity and limitations

Maturity is a complete small product: 33 indexable URLs, shared layout (navbar, footer), SEO infrastructure (`robots.ts`, `sitemap.ts`, `manifest.ts`, JSON-LD), and repository documentation (`README.md`, `AGENTS.md`, `CONTRIBUTING.md`, `SECURITY.md`). Key limitations: (a) all editor capability depends on the third-party iframe and network access to `onecompiler.com`; (b) no offline support, persistence, user accounts, or usage analytics in the repo; (c) no automated tests. **Status:** IMPLEMENTED for (a)–(c); see Sections 25 and 26.

# 2 Project Overview

**Status:** IMPLEMENTED.

### 2.1 Identity

Table 1: Project identity (evidence-based)

| Item | Value |
|---|---|
| Name | CodeLab |
| Subtitle | Online Compiler |
| Repository layout | Next.js App Router application at repository root |
| App package | `codelab`, version `0.1.0`, private, ISC license, author Jay Jogane (`package.json`) |
| Entry route | `/` from `src/app/page.tsx` |
| Document title | "CodeLab — Write. Run. Ship Code." in `src/app/layout.tsx` |
| Remote | `https://github.com/unkown812/codelab.git` (branch `master`) |
| Observed deployment | Serves the application at `https://code-lab.vercel.app/` (verified live on 2026-10-11; no hosting config in repo, so operator-owned) |

### 2.2 Repository inventory

The complete application source (excluding `node_modules/`, `.next/`, `.git/`) is:

- `src/app/` — 17 static routes (`page.tsx` files), 2 dynamic routes (`languages/[slug]`, `blog/[slug]`), plus `layout.tsx`, `globals.css`, `robots.ts`, `sitemap.ts`, `manifest.ts`, `not-found.tsx`.
- `src/components/` — `Navbar`, `Footer`, `CodeEditor` (+ `buildOneCompilerSrc`), `CompilerView` (client island), `LanguageExplorer` (client island), `Breadcrumbs`, `JsonLd`.
- `src/lib/` — `seo.ts` (canonical site config, 12-language metadata), `posts.ts` (4 blog posts).
- `public/` — `favicon.png` (261x261), `favicon.ico` only.
- Root config — `package.json`, `package-lock.json` (npm), `next.config.ts`, `tsconfig.json`, `eslint.config.mjs`, `.gitignore`, `.editorconfig`, `LICENSE` (ISC, Jay Jogane).
- Root docs — `README.md`, `AGENTS.md`, `CONTRIBUTING.md`, `SECURITY.md`, `DESIGN.md`.
- `.github/` — CI workflow, PR template, issue templates, Dependabot config.
- `documents/` — this LaTeX suite (print-oriented; `src/` is the source of truth).

### 2.3 Explicit non-goals (not in repository)

No backend, database, migrations, API endpoints, authentication, authorization, middleware, background jobs, webhooks, storage, analytics, tests, environment files, or Docker images exist. These areas are documented as NOT APPLICABLE in their respective sections rather than omitted silently.

## 3 Product Description

**Status:** IMPLEMENTED for features; INFERRED where user value is interpreted (marked).

### 3.1 Product identity

CodeLab is a free online compiler with editorial-grade guides: a live playground for 80+ languages plus server-rendered content (language guides, tutorials, use cases, comparisons). Product category: online code playground with documentation hub. **Status:** IMPLEMENTED.

### 3.2 Problem and solution

Problem (INFERRED): users who want to try, learn, or share code without installing toolchains need an instantly available editor *and* answers to follow-up questions (stdin handling, sharing, language choice). Solution (IMPLEMENTED): the `/compiler` route fills the viewport with the OneCompiler iframe framed by CodeLab chrome, while 32 companion pages provide guides, FAQs, and comparisons in crawlable HTML.

### 3.3 Target audience and personas

The repository now encodes audiences as first-class pages under `src/app/use-cases/`: students, teachers, and interview candidates, each with a dedicated workflow page. **Status:** IMPLEMENTED (pages) / INFERRED (that these convert to product requirements—no tracker exists).

### 3.4 Features (implemented)

Table 2: Implemented features with evidence

| Feature | Evidence |
|---|---|
| Full-height playground | `/compiler` renders `CompilerView` (runtime switcher pills) over `CodeEditor` in a 72vh studio window |
| OneCompiler embed | `iframe#onecompiler`, `allow="clipboard-read; clipboard-write"`, `loading="lazy"` |
| Configurable embed URL | `buildOneCompilerSrc` supports `language`, `codeId`, `theme`, `fontSize`, 12 boolean flags, plus full `src` override |
| Per-language guides | 12 static pages from `languages/[slug]` with sample programs, steps, FAQs (`src/lib/seo.ts`) |
| Blog guides | 4 posts from `blog/[slug]` with sections, FAQs, related links (`src/lib/posts.ts`) |
| Use-case workflows | `/use-cases` hub + students/teachers/interviews pages |
| Legal + contact | `/about`, `/contact`, `/privacy`, `/terms` (factual, zero-backend wording) |
| SEO infrastructure | `robots.ts`, `sitemap.ts` (33 URLs), `manifest.ts`, per-page canonical/OG/Twitter, JSON-LD via `JsonLd`, breadcrumbs |
| Custom 404 | `not-found.tsx` with navigation links |

### 3.5 Non-features

User accounts, saved sessions, multi-file projects owned by CodeLab, theming beyond the embed `theme`/`fontSize` parameters, and offline execution do not exist in the repository. **Status:** NOT APPLICABLE (not implemented).

### 3.6 Visual identity and SEO keywords

Visual identity follows `DESIGN.md`: near-black canvas (`#0b0b0b`), coral accent (`#f36458`), Inter display type with IBM Plex Mono eyebrows, pill CTAs, `lucide-react` link arrows. Metadata title "CodeLab — Write. Run. Ship Code." and per-route descriptions target "online compiler", "run {language} online", and stdin/sharing how-to queries. **Status:** IMPLEMENTED. See Sections 22–24.

## 4 Business Requirements Document (BRD)

**Status:** INFERRED from repository content (no BRD, tracker, or business metrics exist in the repo). Business facts below trace to implemented pages; anything measurable is marked UNKNOWN, never invented.

### 4.1 Business problem

Learners, teachers, and interview candidates need a zero-install way to run code and get answers, while project owners need a low-cost surface that attracts and helps those users. **Status:** INFERRED from the product shape (free playground + guides + use-case pages).

### 4.2 Business objectives (as evidenced)

1. Offer a free, no-signup compiler playground (`/compiler`). **Status:** IMPLEMENTED.
2. Answer follow-up questions in owned content (language guides, stdin/sharing tutorials, FAQs) instead of losing users to external forums. **Status:** IMPLEMENTED (`/languages`, `/blog`, `/docs`).
3. Address three audiences explicitly (students, teachers, interview prep). **Status:** IMPLEMENTED (`/use-cases/*`).
4. Present tiered plans (Playground / Pro / Classroom on `/pricing`). **Status:** IMPLEMENTED as presentational content only — no checkout, billing, or entitlement logic exists in the repository, so no revenue behavior is claimed.

### 4.3 Stakeholders and user groups

Table 3: Stakeholders (evidence-based)

| **Stakeholder** | **Evidence** |
|---|---|
| Creator/owner | Jay Jogane (`LICENSE`, `package.json` author) |
| Learners, teachers, candidates | Dedicated `/use-cases/*` pages and guide content |
| Hosting/runtime vendors | Vercel (observed host), OneCompiler (embed provider) — third parties, not stakeholders of record |
| Paying customers, partners | UNKNOWN — none evidenced in the repo; pricing tiers are content, not contracts |

### 4.4 Success metrics, constraints, risks

Success metrics: UNKNOWN — no analytics, targets, or KPIs exist in the repo (suggested starting points, RECOMMENDED: playground runs per week, guide-to-compiler conversion, all measurable only after adding privacy-respecting analytics). Constraints: zero-backend architecture (no accounts, no payments, no persistence); ISC open-source license; total dependence on the

OneCompiler runtime. Risks: embed provider changes break the core value without a CodeLab code change; pricing content promises tiers the codebase cannot enforce. See Sections 27 and 25.

# 5 Product Requirements Document (PRD)

**Status:** INFERRED from implementation (no PRD exists in the repository). Nothing below should be read as a historical requirement; it reconstructs what the code actually satisfies.

## 5.1 Vision and goals

Vision (INFERRED): the fastest path from link to runnable code in a browser, with answers attached. Goals satisfied by the code: (1) runnable playground with no signup (`/compiler`); (2) every further question answered in crawlable pages (languages, stdin, sharing, use cases); (3) zero CodeLab-side code execution to operate.

## 5.2 Personas and user stories

Personas are encoded as use-case pages, not as repo requirements. **Status:** IMPLEMENTED (pages) / UNKNOWN (tracker). Reconstructed stories (INFERRED): as a student, I can run Python online without installing anything (`/languages/python`); as a teacher, I can share one link the whole room runs (`/use-cases/teachers`); as a candidate, I can drill with stdin inputs (`/use-cases/interviews`); as an embedder, I can pass `language`/`fontSize`/hide-flags to tailor the iframe per page.

## 5.3 Scope

In scope (IMPLEMENTED): playground with runtime switcher, 12 language guides, 4 blog guides, 3 use-case pages, 1 comparison (+ hub), legal/contact/about pages, SEO/AEO/GEO infrastructure, CI lint+build gates. Out of scope (NOT APPLICABLE): accounts, persistence, collaboration, grading, plagiarism detection, admin tooling.

## 5.4 Acceptance criteria (verifiable)

1. `npm run build` succeeds with all 33 indexable URLs prerendered (verified 2026-10-11). **Status:** IMPLEMENTED.
2. `/compiler` renders the runtime switcher plus one `iframe#onecompiler` whose `src` changes with the selected language. **Status:** IMPLEMENTED.
3. Every indexable route exposes a unique title, description, canonical URL, and breadcrumb trail (verified by code inspection). **Status:** IMPLEMENTED.
4. With network access to `onecompiler.com`, the iframe loads the editor; without it, the page frame still renders. **Status:** INFERRED (third-party behavior, not mocked in repo).

## 5.5 Constraints, dependencies, risks

Constraint: all execution capability is delegated to OneCompiler; CodeLab cannot fix editor bugs or guarantee language availability. Dependency: public internet + `onecompiler.com` availability. Risk: embed policy or URL-schema changes break the playground without a CodeLab code change. Mitigation (RECOMMENDED): pin and document a known-good embed URL and add a smoke check (see Section 27).

# 6 Functional Requirements

**Status:** IMPLEMENTED unless marked otherwise. IDs are documentation-only (no tracker exists in the repo).

Table 4: Functional requirements traceable to code

| ID | Requirement | Evidence |
|---|---|---|
| FR-01 | Render a landing page with a live playground preview on `/` | `page.tsx` hero + studio-window preview embedding `CodeEditor` |
| FR-02 | Embed the OneCompiler editor via iframe | `CodeEditor.tsx`, `iframe#onecompiler` |
| FR-03 | Default embed hides the remote title and enables code-change events | Defaults `hideTitle=true`, `codeChangeEvent=true` |
| FR-04 | Allow language pre-selection via URL path | `language` prop → `/embed/{language}` |
| FR-05 | Allow preloading saved code via code id | `codeId` prop → `/embed/{language}/{codeId}` |
| FR-06 | Allow editor theme and font-size selection | `theme`, `fontSize` (clamped 8–32 in `clampFontSize`) |
| FR-07 | Allow hiding editor chrome elements | Flags `hideNew`, `hideRun`, `hideStdin`, `hideResult`, etc. |
| FR-08 | Allow full URL override | `src` prop bypasses the builder |
| FR-09 | Display static window title and status | `title` default "Code Lab Compiler", `statusText` default "Ready to compile" |
| FR-10 | Hide status cluster on narrow screens | `max-sm:hidden` on status container |
| FR-11 | Provide a full-height playground with runtime switcher on `/compiler` | `CompilerView.tsx` (`"use client"`, language pills, `key` remount per language) |
| FR-12 | Serve 12 per-language guides with samples, steps, FAQs | `languages/[slug]` + `LANGUAGE_META`, `SAMPLES` in code; `generateStaticParams` |
| FR-13 | Serve 4 blog guides with sections, FAQs, related links | `blog/[slug]` + `POSTS` in `src/lib/posts.ts` |
| FR-14 | Serve use-case, comparison, legal, and info pages | `use-cases/*`, `compare/*`, `about`, `contact`, `privacy`, `terms`, `features`, `pricing`, `docs` |
| FR-15 | Expose sitemap, robots, and web manifest | `sitemap.ts` (33 URLs), `robots.ts`, `manifest.ts` |
| FR-16 | Emit structured data and breadcrumbs on content pages | `JsonLd.tsx`, `Breadcrumbs.tsx` (WebSite, WebApplication, FAQPage, HowTo, BlogPosting, BreadcrumbList) |
| FR-17 | Render a helpful 404 with navigation | `not-found.tsx` (home/compiler/docs links) |
| FR-18 | Filter the language index client-side | `LanguageExplorer.tsx` search over `LANGUAGE_SLUGS` |

Out of scope: code persistence, user-triggered run from CodeLab chrome (the component only

forwards `listenToEvents` intent to the embed; no `postMessage` sender exists in the repo), and any server-side compilation. **Status:** NOT APPLICABLE.

# 7 Non-Functional Requirements

Table 5: Non-functional requirements and their implementation state

| **Category** | **Statement** | **Status and evidence** |
|---|---|---|
| Responsiveness | Usable from 360px phones to widescreen; grids collapse, type steps down | IMPLEMENTED: collapsing grids, mobile menu, `next/image` dimensions |
| Performance | First paint dominated by framework + one iframe | INFERRED: 33 static/prerendered URLs, `loading="lazy"` on iframe, two tiny client islands; no measurements in repo |
| Accessibility | Labelled landmarks, skip link, visible focus, keyboard-native controls | PARTIAL: implemented as listed; remote iframe content and smallest muted labels excepted |
| Compatibility | Modern browsers with iframe + clipboard permissions | IMPLEMENTED: standard iframe attributes; no polyfills in repo |
| Security | No CodeLab-side secret handling or code execution | IMPLEMENTED: no secrets, no API routes; residual iframe risks in Section 17 |
| Maintainability | Typed props, tokenized styling, content data in `lib` | IMPLEMENTED: `CodeEditor.tsx` + `buildOneCompilerSrc`; `seo.ts`/`posts.ts` datasets |
| Availability | Page frames render even if embed is unreachable | INFERRED: no error boundary exists; frame renders, iframe area stays empty (RECOMMENDED improvement) |

# 8 Technical Requirements

**Status:** IMPLEMENTED (toolchain table verified against manifests on 2026-10-11).

Table 6: Technology stack (actual versions)

| Category | Technology | Evidence |
|---|---|---|
| Framework | Next.js 16.4.0 (App Router) | `package.json`; `next.config.ts` enables `cacheComponents`, partial prefetching |
| UI runtime | React 19.3.0, react-dom 19.3.0 | `package.json` |
| Language | TypeScript 5, strict mode | `tsconfig.json` (`strict:true`, `jsx:react-jsx`, path alias `@/*`) |
| Styling | Tailwind CSS 4 via `@tailwindcss/turbopack` | `package.json` devDependencies; `globals.css` `@import "tailwindcss"` + `@theme` tokens |
| Icons | `lucide-react` 1.55.0 | `package.json`; link arrows sitewide |
| Fonts | Inter + IBM Plex Mono via `next/font` | `layout.tsx` |
| Lint | ESLint 9 + `eslint-config-next` | `eslint.config.mjs`, script `npm run lint` |
| Runtime embed | OneCompiler iframe embed | `CodeEditor.tsx` `EMBED_BASE` |
| Package metadata | ISC license, author Jay Jogane | `package.json` `license/author` matching `LICENSE` |
| CI | GitHub Actions lint+build | `.github/workflows/ci.yml` (Node 22, `npm ci`) |

### 8.1 Explicitly absent

Table 7: Technologies not present (verified by file search)

| Area | Status |
|---|---|
| Backend / API routes | NOT APPLICABLE — no `route.ts`, no server actions |
| Database / ORM / migrations | NOT APPLICABLE |
| Authentication / sessions | NOT APPLICABLE |
| State management library | NOT APPLICABLE (props + two `useState` islands) |
| Test runner | NOT APPLICABLE (no test files, scripts, or config) |
| Docker / IaC | NOT APPLICABLE |
| Env configuration | NOT APPLICABLE (no `.env*` files; `.gitignore` merely ignores them) |

## 9 Software and System Architecture

**Status:** IMPLEMENTED. This section merges system architecture, high-level design, and low-level design because the system is small enough that separate documents would repeat each other.

### 9.1 System context

The deployed unit is a set of static Next.js routes served to a browser; the browser then loads the OneCompiler application in an iframe on the playground routes. CodeLab executes no user code itself.

Listing 1: System context (text diagram; Mermaid source in diagrams/system-context.mmd)

```
Browser -> Next.js static routes (layout + pages + CodeEditor chrome)
Browser -> https://onecompiler.com/embed... (iframe, editor + runner)
```

```
CodeLab server -x- user code (never touches it)
```

### 9.2 High-level design

1. **Route layer:** `src/app/layout.tsx` (html/body shell, fonts, metadata, navbar/footer) composes 17 static routes and 2 dynamic route segments.
2. **Component layer:** server-rendered pages; exactly two client islands (`CompilerView`, `LanguageExplorer`) own the only `useState` in the app.
3. **Content-data layer:** `src/lib/seo.ts` (site config, 12-language metadata) and `src/lib/posts.ts` (4 blog posts) feed the programmatic routes.
4. **URL-builder layer:** `buildOneCompilerSrc` maps typed props to an embed path plus query string; `src` bypasses it.
5. **SEO layer:** per-route metadata, `JsonLd` structured data, `Breadcrumbs`, `sitemap.ts`, `robots.ts`, `manifest.ts`.
6. **Presentation layer:** Tailwind utilities over `@theme` design tokens in `globals.css`; `lucide-react` icons.

### 9.3 Low-level design

`CodeEditor` is a React Server Component (no `"use client"` directive) with no hooks, context, or event listeners. Data flow is one-way: props → `iframeSrc` string → `iframe src` attribute. `clampFontSize` enforces the documented 8–32 range. `language`/`codeId` are sanitized by stripping `/` characters before interpolation. Dynamic routes use Next.js 16 async `params` with `generateStaticParams` over the `lib` datasets, so all 33 indexable URLs prerender at build time. There are no classes, stores, or services.

### 9.4 Backend, database, and API

**Status:** NOT APPLICABLE. No server components fetch data, no route handlers exist, and no schema or migration files are present. Any future backend must be documented separately; this suite does not invent one.

## 10 Frontend Architecture

**Status:** IMPLEMENTED.

### 10.1 Route tree

17 static routes (`/`, `about`, `blog`, `compiler`, `compare` + article, `contact`, `docs`, `features`, `languages`, `pricing`, `privacy`, `terms`, `use-cases` + 3), 2 dynamic segments (`languages/[slug]` ×12, `blog/[slug]` ×4), shared `layout.tsx` (navbar, footer, skip link), `not-found.tsx`, and metadata routes (`robots.ts`, `sitemap.ts`, `manifest.ts`). Route types are generated via `next typegen` (`LayoutProps<"/">` in `layout.tsx` reflects Next.js 16 conventions).

### 10.2 Component catalogue

Table 8: Components (exhaustive)

| Component | Responsibility |
|---|---|
| `RootLayout` (`layout.tsx`) | html lang, Inter/Plex Mono variables, full `metadata` (title template, canonical, OG/Twitter, robots), skip link, navbar/footer shell |
| `Navbar` | Logo (`next/image` favicon + wordmark + brand dot), route links with active state, mobile menu (`"use client"`) |
| `Footer` | 6-column link grid, copyright + brand lockup |
| `CodeEditor` | Window header (dots, title, status) + responsive iframe body; URL builder |
| `CompilerView` | Runtime switcher pills + remounted `CodeEditor` per language (`"use client"`) |
| `LanguageExplorer` | Search filter over the 12 pinned languages (`"use client"`) |
| `Breadcrumbs` + `JsonLd` | Visible trail plus `BreadcrumbList` schema; generic JSON-LD injector |

### 10.3 Props contract

`CodeEditorProps` exposes `title`, `statusText`, `id`, `className`, `src`, `language`, `codeId`, `theme`, `fontSize`, and 12 embed flags. Defaults reproduce the legacy prototype URL (`theme=dark`, `hideTitle`, `codeChangeEvent`). Note an inconsistency documented in Section 25: `buildOneCompilerSrc` defaults `availableLanguages` and `listenToEvents` to `true` while the component defaults them to `false`, so direct helper calls and default component renders produce different query strings.

### 10.4 Styling approach

Utility-first Tailwind classes over `@theme` tokens (`bg-canvas`, `bg-brand`, `text-ash`, `rounded-marketing`, `rounded-app-*`); no CSS modules. Icons are `lucide-react` only (`ArrowRight`/`ArrowDown`, `aria-hidden`, sized to text). `next/image` serves the favicon logo.

### 10.5 State and data fetching

Two `useState` islands only (`CompilerView` language, `LanguageExplorer` query). No context, no fetch, no client cache beyond framework defaults in `next.config.ts` (`cacheComponents`, partial prefetching).

## 11 Integrations

**Status:** IMPLEMENTED (one integration).

### 11.1 OneCompiler embed

Table 9: OneCompiler integration fact sheet

| Item | Detail |
|---|---|
| Type | Third-party iframe embed; editor and code runner hosted by OneCompiler |
| Base URL | `https://onecompiler.com/embed` (`EMBED_BASE` in `CodeEditor.tsx`) |
| DOM contract | `iframe#onecompiler`, `title="Online Code Compiler"`, `loading="lazy"`, `allow="clipboard-read; clipboard-write"` |
| Configuration surface | Path (`language`, `codeId`) + query flags (Section 6, FR-04–FR-08) |
| Programmatic API | Only intent flags today: `codeChangeEvent` (embed posts code changes) and `listenToEvents` (embed accepts incoming messages). No `window.onmessage` receiver and no `postMessage` sender exist in this repo. |
| Docs reference | `https://onecompiler.com/apis/embed-editor` (public embed options) |

### 11.2 Resilience notes

If `onecompiler.com` is unreachable, the CodeLab chrome still renders with an empty editor area; no fallback message or retry exists. **Status:** IMPLEMENTED (behavior) / RECOMMENDED (fallback UI, see Section 27). No other integrations (analytics, auth providers, storage, email) exist. **Status:** NOT APPLICABLE.

## 12 User Flows

**Status:** IMPLEMENTED (flows reconstructed from routes and components; no analytics verify real-user paths).

### 12.1 Flow 1 — run code immediately

Landing (`/`) → "Start coding" → `/compiler` → pick a runtime pill → type code → press Run in the embed → read output in the result pane. Evidence: `src/app/page.tsx` hero CTAs, `CompilerView.tsx` switcher, `CodeEditor.tsx` iframe. Mermaid source: `diagrams/user-flow.mmd`.

### 12.2 Flow 2 — learn a language via a guide

Language index (`/languages`, optional search filter) → per-language guide (`/languages/[slug]`) → copy the sample program → "Run {language} now" → `/compiler` → paste, fill stdin if needed, Run. Evidence: `LanguageExplorer.tsx`, `SAMPLES` and steps in the slug template.

### 12.3 Flow 3 — teacher assigns, student submits by link

Teacher opens `/use-cases/teachers` → shares one starter compiler link → students edit and re-share their run URLs → teacher opens each link to review exact code and inputs. Evidence: use-case page steps; sharing mechanics in `/docs` and `/blog/share-code-run-link`.

### 12.4 Flow 4 — answer-seeking and support

Docs FAQ, blog guides, and comparison pages cross-link each other and end in compiler CTAs; unresolved questions land on `/contact`, which routes to docs/compiler/about rather than a form (no support desk exists). Evidence: `docs/page.tsx` further-reading block, `contact/page.tsx`.

Listing 2: User flows (text diagram; Mermaid source in diagrams/user-flow.mmd)

```
Visitor -> / -> /compiler -> pick runtime -> type -> Run (embed) -> output
Learner -> /languages -> /languages/[slug] -> copy sample -> /compiler -> Run
Teacher -> use-cases/teachers -> share starter link -> students re-share URLs
Reader -> docs/blog/compare -> related links -> compiler CTA or /contact
```

## 13 Data Flows

**Status:** IMPLEMENTED. CodeLab stores nothing; data moves at build time (content) and at runtime (browser to third-party embed).

### 13.1 Build-time data flow

Content datasets in `src/lib/` (`seo.ts` language metadata, `posts.ts` blog posts) feed `generateStaticParams`, which prerenders 12 language guides and 4 blog posts; `sitemap.ts` enumerates the same sources, so sitemap and pages cannot drift as long as both read the datasets. Mermaid source: `diagrams/data-flow.mmd`.

Listing 3: Build-time data flow (text diagram; Mermaid source in diagrams/data-flow.mmd)

```
src/lib/seo.ts + posts.ts -> generateStaticParams -> 16 static pages
src/lib/seo.ts -> sitemap.ts -> sitemap.xml (33 URLs)
props -> buildOneCompilerSrc -> iframe src (runtime)
```

### 13.2 Runtime data flow

1. **Playground:** user keystrokes stay in the browser; on Run, code and stdin travel directly from the browser to `onecompiler.com` inside the cross-origin iframe. CodeLab servers see nothing (there are no servers beyond static hosting).
2. **UI state:** two local `useState` values (selected runtime, search query) never leave the page and are never persisted.
3. **Sharing:** run state travels peer-to-peer inside share URLs; CodeLab stores no snippets, accounts, or histories.

### 13.3 Third-party data exposure

The only data leaving the browser goes to the embed provider on user-initiated Run, governed by OneCompiler's policies, plus standard web logs at the static host. No analytics, cookies, or tracking pixels are set by CodeLab code. **Status:** IMPLEMENTED (verified by dependency and code inspection; see Section 17).

## 14 Configuration

**Status:** IMPLEMENTED.

Table 10: Configuration files and their role

| File | Role |
|---|---|
| `next.config.ts` | Enables `cacheComponents` and partial prefetching; Turbopack rule routing `*.css` through `@tailwindcss/turbopack` |
| `tsconfig.json` | Strict TS, `ES2017` target, bundler resolution, `@/*` → `./src/*` alias |
| `eslint.config.mjs` | Flat config extending `core-web-vitals` + TypeScript presets; ignores build outputs |
| `.gitignore` | Ignores `node_modules`, `.next`, `out`, `build`, `.env*`, Vercel metadata, `*.tsbuildinfo` |
| `.editorconfig` | LF endings, UTF-8, 2-space indent, final newline |
| `package.json` | Scripts `dev/build/start/lint`; `license` ISC + `author` Jay Jogane matching `LICENSE` |
| `.github/workflows/ci.yml` | CI: `npm ci` → lint → build on Node 22 (see Section 21) |

### 14.1 Environment variables

None are read or required. **Status:** NOT APPLICABLE. The table below records the empty set explicitly so Word readers are not left guessing.

Table 11: Environment variables

| Variable | Required | Used by | Sensitive |
|---|---|---|---|
| (none) | — | — | — |

## 15 UI/UX

**Status:** IMPLEMENTED.

### 15.1 Screens

33 indexable pages sharing one shell: sticky dark navbar (logo, route links with active state, mobile hamburger menu), content sections alternating dark/light polarity, and a 6-column footer. Key screens: landing with playground preview (`/`), full-height playground (`/compiler`), searchable language index, 12 language guides, pricing tiers, docs with FAQ accordions, blog, use cases, comparison, legal pages, and a custom 404 with navigation links.

### 15.2 Layout and responsive behavior

Content pages center in a `max-w-[1640px]` container with 96px section rhythm; display type steps down at small breakpoints (112px → 64px hero). The editor regions use fixed heights owned by parents (`h-[560px]` preview, `72vh` playground) with the section at `h-full`. Mobile menu collapses under 1024px; the pricing grid collapses 3-up to 1-up; footer collapses 6-column to stacked. `next/image` serves the logo with explicit dimensions (no layout shift).

### 15.3 States and validation

Interactive states owned by CodeLab: runtime switcher pills (`aria-pressed`), search filter with an explicit empty state ("No match" + pointer to the compiler picker), FAQ `details/summary` disclosures, mobile menu toggle (`aria-expanded`). The embed `statusText` ("Ready to compile")

remains static text, not a live reflection of embed readiness. **Status:** IMPLEMENTED (static status) / RECOMMENDED (drive it from embed events or remove it to avoid implying liveness).

### 15.4 Accessibility

Implemented: skip-to-content link, `:focus-visible` outlines, landmark `section` elements with `aria-label`, descriptive iframe `title`, decorative dots hidden via `aria-hidden`, labelled search input, keyboard-native disclosures. Not implemented: keyboard control of remote iframe content (impossible cross-origin) and AA contrast on the smallest muted mono labels. **Status:** PARTIAL (see Section 25).

## 16 Design System

**Status:** IMPLEMENTED. Tokens are defined in `DESIGN.md` (Sanity-inspired) and implemented as Tailwind v4 `@theme` tokens in `src/app/globals.css`. Values below verified against both files on 2026-10-11.

Table 12: Actual design values in use

| Token | Value and location |
|---|---|
| Canvas | `#0b0b0b` dark sections (`bg-canvas`); `#ffffff` light commercial sections; `#ededed` paper bands |
| Accent | Coral-red `#f36458` (`bg-brand`), reserved for one primary action per viewport + brand dot beside the wordmark |
| Text | White on dark; ink `#0b0b0b` on light; ash `#b9b9b9` body copy on dark; mute `#797979` eyebrows |
| Typography | Inter (display/body, tight negative tracking) + IBM Plex Mono (eyebrows/labels only) via `next/font`; display 112px → 38px scale |
| Radius | Pill `99999px` marketing CTAs; 3–6px `rounded-app-*` for inputs and studio windows; 12px `rounded-marketing` cards |
| Icons | `lucide-react` only (`ArrowRight`/`ArrowDown`), `aria-hidden`, sized to surrounding text |
| Section rhythm | 96px verticals; 1640px max container; dark/light polarity flips as dividers |

Editor chrome (framed screenshots, traffic-light dots, status pill) follows the same tokens with native-app radii so embedded product UI reads as product, not marketing.

## 17 Security

**Status:** findings are IMPLEMENTED observations; recommendations are marked RECOMMENDED. Reporting policy lives in `SECURITY.md` (private GitHub Security Advisories).

Table 13: Security review (evidence-based)

| Finding | Severity | Evidence, impact, recommendation |
|---|---|---|
| No secrets in repo | Informational | No `.env` files, tokens, or keys found (re-scanned 2026-10-11). Impact: none. Recommendation: keep it so; `CONTRIBUTING.md` forbids committing secrets. |
| Cross-origin iframe | Medium | `iframe` loads full OneCompiler app. Impact: CodeLab inherits the embed's XSS/sandbox posture; CodeLab cannot audit remote JS. Recommendation (RECOMMENDED): add `sandbox` allow-list review and a content-security-policy frame-src pin in future hosting config. |
| Clipboard permissions | Low | `allow="clipboard-read; clipboard-write"`. Impact: editor copy/paste works; a compromised embed could read clipboard on user gesture. Recommendation: keep only if needed; document the trade-off. |
| No input validation surface | Low | CodeLab takes no user input itself (only props + two local UI states). Impact: injection risk is confined to `language`/`codeId` interpolation, which strips slashes. Recommendation: retain the sanitization; add unit tests for the builder. |
| Missing security headers | Low | No `headers()` config in `next.config.ts`. Impact: hosting defaults apply. Recommendation (RECOMMENDED): set `frame-ancestors`, `X-Content-Type-Options` at deploy time. |
| Shared-link data exposure | Low | Run state travels in share URLs (documented on `/privacy`). Impact: secrets pasted into shared code leak to link holders. Recommendation: keep the documented warning; no code change needed. |

Authentication, authorization, RLS, encryption, upload handling, and audit logging are NOT APPLICABLE: the features do not exist.

# 18 Performance and Scalability

**Status:** INFERRED (no measurements, budgets, or load tests exist in the repository).

## 18.1 Performance

Each page ships static HTML plus the React runtime plus, on playground routes, one lazily-loaded third-party iframe. Expected cost centers: (1) Next.js JS bundle, (2) Inter/Plex Mono webfont loading via `next/font`, (3) the OneCompiler app inside the iframe, which dominates interactive readiness. Implemented mitigations: static prerendering of all 33 URLs, `loading="lazy"` on iframes, explicit `next/image` dimensions for logos, two minimal client islands, no analytics or ad

scripts. Not implemented: bundle analysis, `next/image` beyond logos (no photo assets exist), or Web Vitals reporting.

### 18.2 Scalability

CodeLab-side scaling is trivially horizontal: the routes are static with no session state, so any static host or CDN can serve them. Effective scalability is bounded by OneCompiler's availability and rate limits, which are outside this repository. No caching, queueing, or database scaling considerations apply. **Status:** NOT APPLICABLE beyond CDN static hosting.

## 19 Testing

**Status:** NOT APPLICABLE (no test infrastructure exists) — documented explicitly rather than invented.

### 19.1 Current state

No test runner, script, or spec file is present: `package.json` defines only `dev/build/start/lint`; file search finds no `*.test.*` or `*.spec.*`; no Playwright/Cypress/Vitest/Jest configuration exists. Automated quality gates are `eslint`, the TypeScript check inside `next build`, and the CI workflow (all passing at the time of analysis, 2026-10-11).

### 19.2 Recommended strategy (RECOMMENDED, not implemented)

1. **Unit:** tests for `buildOneCompilerSrc` (path joining, flag serialization, `fontSize` clamping, slash stripping) and the helper/component default mismatch noted in Section 25.
2. **Component:** render test asserting the compiler region contains one iframe with the correct `src` for given props.
3. **E2E smoke (optional):** visit `/compiler`, assert the frame renders; stub or allow-list the third-party iframe per environment policy.
4. **Manual QA checklist:** viewport check at 360/768/1440px; embed loads with network; graceful empty editor area without network; clipboard copy/paste inside the embed; mobile menu and search filter behavior.

## 20 Deployment

**Status:** IMPLEMENTED for local commands and CI; INFERRED for hosting (no hosting config in repo).

### 20.1 Prerequisites

Node.js with npm (versions not pinned in repo — RECOMMENDED: pin in `.nvmrc` or `packageManager` field; CI currently builds on Node 22). Install from the repository root:

Listing 4: Verified commands (from package.json scripts)

```
npm install
npm run dev    # local development
npm run lint   # quality gate
npm run build  # production build + type check
npm start      # serve the production build
```

### 20.2 Build and hosting

`next build` prerenders all routes (verified 2026-10-11 output: static + partial-prerender entries, dynamic `sitemap.xml`). CI (`.github/workflows/ci.yml`) runs install, lint, and build on pushes to `master` and pull requests. The app was observed serving at `https://code-lab.vercel.app/` on 2026-10-11, consistent with Vercel hosting, but no platform manifest exists in the repo, so hosting remains an operator choice. No database migrations, rollback scripts, or production env configuration exist. **Status:** NOT APPLICABLE.

## 21 CI/CD

**Status:** IMPLEMENTED (workflow file present; run history not verifiable from the repo).

### 21.1 Continuous integration

`.github/workflows/ci.yml` runs on pushes to `master` and all pull requests: checkout, Node 22 setup with npm cache, `npm ci`, `npm run lint`, `npm run build` (which includes the TypeScript check). Permissions are minimal (`contents: read`). There is no test job because no test suite exists (see Section 19).

### 21.2 Supporting automation

`.github/dependabot.yml` schedules weekly npm updates grouped for Next.js and React. PR and issue templates (`.github/PULL_REQUEST_TEMPLATE.md`, `ISSUE_TEMPLATE/`) enforce the lint/typecheck/build checklist. **Status:** IMPLEMENTED. No deployment workflow exists; hosting remains operator-owned (see Section 20). **Status:** NOT APPLICABLE.

## 22 SEO

**Status:** IMPLEMENTED (verified in code on 2026-10-11). AEO and GEO have dedicated implementations in Sections 23 and 24.

### 22.1 Current implementation

Table 14: SEO signals present in the repo

| Signal | Evidence |
|---|---|
| Title template + descriptions | `layout.tsx` title template `%s -- CodeLab`; unique title/description per route |
| Canonical URLs | `alternates.canonical` on every route; `metadataBase` set to the production URL |
| Open Graph + Twitter | Per-route OG/Twitter titles, descriptions, images (`layout.tsx` + page `metadata`) |
| Robots + sitemap + manifest | `robots.ts`, `sitemap.ts` (33 URLs), `manifest.ts` |
| Crawlable content | Server-rendered guides, FAQs, comparisons; editor-only content avoided for key facts |
| Semantic HTML | One H1 per route, breadcrumb `nav`, tables with `th scope`, labelled landmarks |
| Internal linking | Navbar, footer grid, contextual cross-links; no orphan indexable route |
| Custom 404 | `not-found.tsx` with navigation (no soft-404) |

### 22.2 Known limitations

The playground editor itself lives in a cross-origin iframe and contributes no crawlable text; surrounding pages carry the indexable content instead. The social/OG image reuses the 261px favicon (no dedicated 1200px share image). **Status:** IMPLEMENTED (limitation) / RECOMMENDED (dedicated OG image, see Section 27).

## 23 AEO (Answer Engine Optimization)

**Status:** IMPLEMENTED (verified in code on 2026-10-11).

### 23.1 Answer-first content

Key facts (what CodeLab is, who it is for, whether it is free) exist as plain crawlable HTML, never hidden exclusively behind JavaScript interactions: the home page carries a "What is CodeLab? / Who is it for? / Is it free?" answer block; every language guide and blog post repeats its FAQs in server-rendered `details/summary` elements paired with `FAQPage` JSON-LD; procedural pages pair visible steps with `HowTo` JSON-LD (`/compiler`, `/docs`, language guides).

### 23.2 Question coverage

The content answers, verbatim in text: what the product is, what it does, who it is for, what problem it solves, how to use it (3-step HowTos), whether it is free (pricing + answers), which languages/versions are pinned, who created it (`/about`), and where to get help (`/contact`). **Status:** IMPLEMENTED. Marketing language is kept out of answer blocks; limitations (no backend, no accounts) are stated plainly so answer engines quote accurate constraints.

## 24 GEO (Generative Engine Optimization)

**Status:** IMPLEMENTED (verified in code on 2026-10-11).

### 24.1 Entity graph

JSON-LD on every route establishes typed entities and their relationships: `WebSite` (CodeLab) → `Person` (creator Jay Jogane, `/about`) → `WebApplication` (the compiler, DeveloperApplication) → `Blog`/`BlogPosting`, `HowTo`, `FAQPage`, `BreadcrumbList`, `AboutPage`, `ContactPage`. No ratings, reviews, offers beyond the free tier, or organizational claims are asserted—every property traces to visible page content.

### 24.2 Citation-friendly properties

Statistics-free, superlative-free prose; code samples and version pins (Python 3.13, Node 24, Java 21) are concrete and quotable; the comparison page states where CodeLab does *not* fit, giving generative engines balanced sourcing. **Status:** IMPLEMENTED. No AI-crawler blocking exists in `robots.ts`; no ranking or citation guarantees are claimed.

## 25 Known Issues

**Status:** IMPLEMENTED observations (re-verified in code on 2026-10-11). Items fixed since v1.0 are marked FIXED, not deleted, so the history stays auditable.

Table 15: Verified issues

| Issue | Severity | Location, impact, recommendation |
|---|---|---|
| Helper/component default mismatch | Medium | `CodeEditor.tsx`: `buildOneCompilerSrc` defaults `availableLanguages=true`, `listenToEvents=true`; the component defaults both to `false`. Impact: documentation examples calling the helper directly produce a different URL than a default render. Recommendation: align the defaults and add unit tests. |
| Static status text | Low | `statusText="Ready to compile"` never changes. Impact: implies liveness the page does not have. Recommendation: drive from embed events or relabel as static chrome. |
| No embed fallback | Medium | No error UI when `onecompiler.com` is unreachable. Impact: blank editor area. Recommendation: detect iframe failure and show guidance. |
| Muted mono contrast | Low | Eyebrow labels in `#797979` on `#0b0b0b` sit below WCAG AA for small text. Impact: hardest-to-read labels are the smallest. Recommendation: lighten token or restrict mute to large/bold use. |
| Manual favicon `<link>` | Low | FIXED 2026-10-11: layout uses `metadata.icons` only. |
| Unused template styling/assets | Low | FIXED 2026-10-11: `globals.css` is token-only; `public/` holds favicons only. |
| OG/manifest icon dimensions | Low | FIXED 2026-10-11: metadata declared 512px but `favicon.png` is 261x261; declarations corrected to actual dimensions. |

## 26 Technical Debt

**Status:** IMPLEMENTED observations; priority is the author's judgment (marked).

Table 16: Technical debt register

| Item | Priority | Detail |
|---|---|---|
| Untested URL builder | High | `buildOneCompilerSrc` is the only logic in the app and has no tests. Pay down with unit tests (Section 19). |
| Default mismatch (Sec. 25) | High | Same fix as above; decide the canonical defaults once. |
| No pinning of embed contract | Medium | Embed URL schema is assumed from public docs; no version pin or smoke test. |
| Unpinned Node runtime | Medium | No `engines`, `.nvmrc`, or `packageManager` field; CI builds on Node 22 by convention only. |
| Missing headers/CSP | Low | No security-header configuration (Section 17). |
| Prototype drift | — | RESOLVED 2026-10-11: the legacy `index.html` and `codelab/` subdirectory no longer exist; this suite now documents the root-level app. |
| Dead template code | — | RESOLVED 2026-10-11 (see Section 25). |

# 27 Roadmap

**Status:** all items RECOMMENDED or POTENTIAL unless marked EXISTING. Nothing below is a committed plan (no tracker exists in the repo).

## 27.1 Existing (EXISTING)

Embed playground with runtime switcher, configurable OneCompiler URL builder, 12 language guides, 4 blog guides, use-case/comparison/legal pages, SEO/AEO/GEO infrastructure (canonicals, OG, sitemap, JSON-LD, breadcrumbs, custom 404), lint+typecheck+CI gates, tokenized design system with icons, pruned template dead code, metadata-only favicon.

## 27.2 Recommended next (RECOMMENDED)

1. Align `buildOneCompilerSrc` and component defaults; add unit + render tests.
2. Add an embed-failure fallback message with retry guidance.
3. Set security headers/CSP at deploy time.
4. Add a dedicated 1200px OG share image (favicon is reused today).
5. Pin Node/npm versions (`.nvmrc` or `packageManager`) and document them.
6. Pin a known-good embed URL in documentation and add a periodic smoke check against `https://onecompiler.com/apis/embed-editor`.
7. Lift muted mono labels to AA contrast.

## 27.3 Potential future work (POTENTIAL)

Optional, only if product direction demands: `postMessage` bridge exposing code-change/run events to CodeLab chrome, multi-tab editors, accounts/persistence (requires a backend design

doc first), and a self-hosted editor to remove the third-party runtime dependency. Each would require new architecture, security, and testing documentation.

# 28 Development Guidelines

**Status:** IMPLEMENTED (workflow as configured). Normative contributor text lives in `AGENTS.md` and `CONTRIBUTING.md`; this section summarizes it for print readers.

## 28.1 Setup and commands

Work from the repository root. Path alias `@/*` maps to `./src/*` (`tsconfig.json`). Run `npm run lint` (zero warnings), `npx tsc -noEmit`, and `npm run build` before opening a change; regenerate route types with `npx next typegen` if `LayoutProps`/`PageProps` go stale. Formatting follows `.editorconfig` (LF, 2-space indent, final newline).

## 28.2 Conventions

- Keep pages server-rendered with `metadata` exports; new interactivity goes in small client components (`CompilerView` and `LanguageExplorer` are the only islands).
- Every new indexable route needs title, description, canonical URL, JSON-LD, and a `sitemap.ts` entry.
- Keep the embed builder as the single place that knows the OneCompiler URL schema; never hand-concatenate embed URLs elsewhere.
- Use Tailwind `@theme` tokens and `lucide-react` icons; never hardcode hex values or invent stats, customers, or offers.
- Content datasets live in `src/lib/` (`seo.ts`, `posts.ts`); extend them instead of ad-hoc JSX.
- Do not add backend, auth, tracking, or dependencies without a design doc and security review.

## 28.3 Definition of done

Build + lint + typecheck green, requirements met, nothing else broken, focused diff reviewed, docs (`README.md`/`AGENTS.md`, plus this suite for behavioral changes) updated, `LICENSE` terms untouched, no secrets added.

# 29 Contribution Guide

**Status:** IMPLEMENTED (summarizes `CONTRIBUTING.md`, `AGENTS.md`, and `.github/` templates; consult those files for the normative text).

## 29.1 Workflow

Setup is `npm install` + `npm run dev`; no services or environment files. Changes must keep `npm run lint` (zero warnings), `npx tsc -noEmit`, and `npm run build` green. One concern per pull request; the PR template requires the verification checklist plus a sitemap entry for any new indexable route.

### 29.2 Conventions for contributors

Follow `DESIGN.md` tokens and `lucide-react` icons; keep pages server-rendered with `metadata`; add content data to `src/lib/` rather than ad-hoc JSX; never invent stats, customers, or offers. `LICENSE` (ISC, Jay Jogane) terms must not change; never force-push or rewrite history. Security reports go through private GitHub Security Advisories per `SECURITY.md`.

## 30 Glossary

Table 17: Glossary (repository-scoped)

| Term | Meaning |
|---|---|
| CodeLab | This project: a free online compiler with guides, built as a static Next.js site. |
| BRD | Business Requirements Document (`brd.tex`); evidence-only, metrics marked UNKNOWN. |
| User flow | A verified visitor journey (run, learn, assign, support) in `user-flows.tex`. |
| Data flow | Build-time and runtime data movement; nothing is stored (`data-flows.tex`). |
| OneCompiler embed | Third-party editor/runner loaded in `iframe#onecompiler` from `onecompiler.com`. |
| `buildOneCompilerSrc` | Helper in `CodeEditor.tsx` mapping props to an embed URL. |
| App Router | Next.js routing model used here (`src/app/` routes, layouts, metadata). |
| Static prerender | Build-time HTML generation; all 33 indexable URLs prerender. |
| `generateStaticParams` | Next.js function pre-rendering dynamic segments from `src/lib/` datasets. |
| JSON-LD | Structured-data format used for WebSite, FAQPage, HowTo, BlogPosting, and breadcrumb schemas. |
| Canonical URL | The single authoritative URL declared per route via `alternates.canonical`. |
| AEO / GEO | Answer-engine / generative-engine optimization: crawlable answers and typed entities for AI search. |
| Client island | The only client-rendered components (`CompilerView`, `LanguageExplorer`); everything else is server-rendered. |
| NOT APPLICABLE | Classification meaning the area does not exist in this repository. |
| INFERRED | Classification meaning a reasonable interpretation of the code, not a repo-stated fact. |
| RECOMMENDED | Classification meaning a suggested improvement, not implemented functionality. |